

13. 온칩 버스 · 인터커넥트 완전 이해 — SoC라는 도시의 도로망

이 문서의 목적. SK하이닉스 설계 JD에 반복적으로 등장하지만("AMBA Bus(AXI, AC)", "On-Chip Interconnector(NoC)", "Top Bus/Async Bridge Implementation", "IP Integration", "Chiplet Interface") 기존 카드·개념서에 정면으로 다룬 자료가 없던 영역 — **온칩 버스/인터커넥트** — 를 1st-principles로 끝까지 세운다. 이건 SoC 설계의 뼈대다. IP를 아무리 잘 만들어도, 그것들을 엮는 도로망을 모르면 "SoC를 설계한다"는 말을 할 수 없다.

지원자 좌표. 삼성 S.LSI에서 RTL 검증·sign-off·SoC 통합환경을 4년 다룬 지원자. 버스/인터커넥트는 두 가지 의미에서 정확히 당신의 무대다 — (1) **IP integration·Top 통합**은 SoC 통합환경 업무 그 자체이고, (2) **버스 프로토콜 검증(UVM VIP + SVA)**은 당신의 가장 강력한 무기다. "AXI를 설계한다"보다 "AXI가 spec대로 도는지 증명한다"가 당신의 강점이며, 그 증명의 대상이 바로 이 문서의 프로토콜이다.

메모리 브릿지 한 문장(미리). "HBM Base Die도, 메모리 컨트롤러도 내부는 결국 IP들이 **버스로 엮인 작은 SoC**입니다. 저는 그 IP들을 통합하고, 그 사이를 흐르는 프로토콜이 spec대로 도는지 UVM/SVA로 증명해 온 사람입니다."

연결. async bridge = CDC 문제 → 깊이는 05_CDC_RDC_완전이해.md . 프로토콜 검증 = UVM/SVA → 07_검증_DFT_MBIST_완전이해.md . off-chip 고속 인터페이스(PCIe/CXL/UCIe PHY) → 09_고속인터페이스_PHY_SIPI_완전이해.md . 이 문서는 **on-chip**(칩 안의 도로망)에 집중하고, 칩 밖으로 나가는 순간 09/10으로 넘긴다.

0. 큰 그림 — 이 문서가 답하는 단 하나의 질문

현대 SoC 안에는 CPU, GPU, DMA, 메모리 컨트롤러, 각종 가속기, 주변장치 컨트롤러 수십~수백 개의 IP가 들어간다. 이들은 서로 다른 회사가, 서로 다른 시기에, 서로 다른 클럭으로 설계했다. 그런데도 칩으로 조립되면 **서로 데이터를 주고받아야** 한다. CPU가 메모리에서 데이터를 읽고, DMA가 그걸 가속기로 옮기고, 가속기가 결과를 다시 메모리에 쓴다.

이 모든 데이터 이동을 가능하게 하는 것이 **온칩 버스/인터커넥트**다. 그리고 이 문서가 끝까지 따라가는 단 하나의 질문은 이것이다:

"준비가 안 맞는 여러 IP 사이로, 누락·중복·교착 없이, 공정하게, 고대역으로 데이터를 흐르게 하려면 어떤 규약과 구조가 필요한가?"

답을 한 칸씩 쌓아 올린다:

IP들이 서로 통신해야 한다 → 공통 인터페이스 규약이 필요하다 (재사용 · 통합)
 → 송수신 준비가 안 맞는다 → valid/ready 핸드셰이크 + backpressure (모든 버스의 원자)
 → 단순 주변장치엔 APB, 파이프라인엔 AHB, 고성능엔 AXI (왜 진화했나)
 → 마스터 · 슬레이브가 여럿이다 → 공유버스 / crossbar / NoC (토폴로지)
 → 동시 요청이 충돌한다 → 중재(arbitration): RR / weighted / credit / QoS
 → 자원이 순환 의존하면 멈춘다 → deadlock/livelock/starvation, 어떻게 막나
 → 버스가 클럭 · 전압 도메인을 건넌다 → async bridge = CDC (05 참조)
 → 이 규약이 지켜지는지 증명한다 → UVM VIP + SVA (07 연결)
 → 이게 전부 HBM Base Die · 컨트롤러 안에서 일어난다 (메모리 맥락)

읽는 순서: §1(왜 규약이 필요한가) → §2(valid/ready 원자) → §3(AMBA 진화) → §4(토폴로지) → §5(중재) → §6(위험: deadlock류) → §7(도메인 가로지르기) → §8(검증) → §9(메모리/HBM) → §10(면접) → §11(자가점검). 각 절은 앞 절의 "왜?"에 답하며 쌓인다.

1. 도입 — SoC는 IP들의 도시, 버스는 도로망

1.1 왜 "공통 인터페이스 규약"이 필요한가

비유부터 박아두자. SoC를 하나의 **도시**라고 하면, 각 IP는 그 도시의 **건물**이다. CPU는 시청, 메모리 컨트롤러는 거대한 창고, DMA는 운송회사, 가속기는 공장이다. 건물만 잘 지어선 도시가 돌아가지 않는다. 건물 사이를 잇는 **도로망**이 있어야 사람과 물자가 오간다. 그 도로망이 버스/인터커넥트다.

그런데 도로에는 약속(규약)이 필요하다. 차선 폭, 신호등 규칙, 우선순위 — 이게 없으면 모든 건물이 자기 멋대로 진입로를 내고 충돌한다. SoC도 똑같다. **공통 인터페이스 규약(common bus protocol)** 이 필요한 진짜 이유는 두 가지다:

1. **IP 재사용(reuse)**. AXI를 따르는 가속기를 한 번 만들어 두면, AXI를 쓰는 어느 SoC에도 그대로 꽂을 수 있다. 만약 IP마다 통신 방식이 제각각이라면, 새 SoC를 만들 때마다 모든 IP의 인터페이스를 새로 손봐야 한다. 표준 규약은 IP를 **레고 블록**으로 만든다. 이게 IP 비즈니스(ARM, 외부 IP 벤더)와 SoC 산업 전체가 굴러가는 토대다.
2. **통합(integration)의 결정론**. SoC 통합 엔지니어(=당신이 하던 일)는 수십 개 IP를 Top에서 엮는다. 모든 IP가 같은 규약을 따르면, 통합은 "포트를 규약대로 연결한다"는 기계적·검증 가능한 작업이 된다. 규약이 없으면 통합은 매번 새로운 번역 작업이 되고, 버그의 온상이 된다.

핵심: **표준 버스 규약은 "IP를 재사용 가능하게 만들고, 통합을 검증 가능하게 만든다."** 이 한 문장이 AMBA가 산업 표준이 된 이유 전체다.

1.2 메모리도 내부는 버스로 엮인 SoC다

여기서 메모리로의 첫 브릿지를 박는다. "메모리는 셀 어레이 아니냐, 버스와 무슨 상관이나"는 오해를 깨야 한다.

- **HBM Base Die**. HBM 스택의 맨 아래 로직 다이(base die)는 사실상 작은 SoC다. 그 안에 TSV로 올라가는 채널 인터페이스, 테스트/리퍼어 로직(07 참조), 그리고 (HBM3/HBM4 세대로 갈수록) 더 많은 디지털 IP가 들어간다. 이것들을 엮는 게 내부 버스/인터커넥트다. JD의 "HBM SoC Design" · "IP Integration"이 정확히 이 지점이다.

- **메모리 컨트롤러.** DRAM 컨트롤러는 호스트 쪽에서 AXI 같은 표준 버스로 요청을 받고(프론트엔드), 내부에서 스케줄링·리오더링·refresh 관리를 한 뒤, DRAM PHY로 명령을 내보낸다(백엔드, 09 참조). 프론트엔드의 AXI 슬레이브 포트, 백엔드로 가는 내부 채널 — 전부 버스 설계다.
- **NAND 컨트롤러/Logic.** 마찬가지로 호스트 인터페이스 ↔ 내부 ECC/FTL 가속 IP ↔ NAND 채널 사이가 버스로 엮인다.

그래서 "메모리 설계 = 셀만 만지는 일"이 아니라, **Peri/로직 영역은 곧 작은 SoC이고, SoC인 이상 버스가 뻗대다.** 당신의 SoC 통합 경험이 메모리로 그대로 이식되는 지점이 바로 여기다.

2. 핸드셰이크의 원자 — valid/ready와 backpressure

모든 온칩 버스를 단 하나의 원자로 분해하면 **2-신호 핸드셰이크(valid/ready handshake)** 가 남는다. AXI도, NoC의 채널도, 잘 설계된 거의 모든 인터페이스가 이 원자 위에 서 있다. 이걸 파형 수준에서 몸에 익히면 나머지는 전부 응용이다.

2.1 문제 설정 — 송신과 수신 of 박자가 안 맞는다

송신자(producer)가 데이터를 만들어 보내려 한다. 수신자(consumer)가 그걸 받아 처리한다. 그런데 이 둘의 속도가 매 사이클 같을 리 없다. 송신이 데이터를 못 만든 사이클도 있고, 수신이 바빠서 못 받는 사이클도 있다. 단순히 "매 클럭 데이터를 던진다"고 하면 못 받는 순간 데이터가 사라진다(loss). 반대로 수신에 같은 데이터를 두 번 받으면 중복(duplication)이다.

이 loss/duplication을 원천 차단하려면, "둘 다 준비됐을 때만, 정확히 그 사이클에만 전송이 일어난다" 는 약속이 필요하다. 그게 valid/ready다.

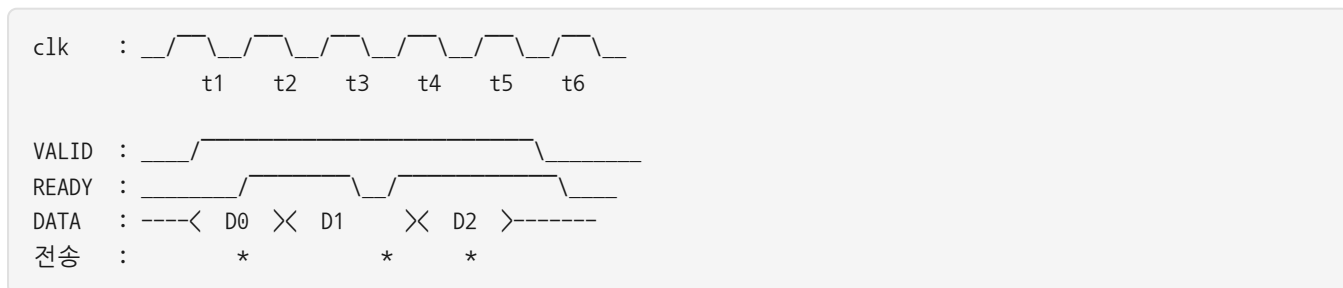
2.2 두 신호의 정의

- **VALID** (송신 → 수신): "내가 줄 데이터가 지금 유효하다." 송신자가 올린다.
- **READY** (수신 → 송신): "나는 지금 받을 수 있다." 수신자가 올린다.
- **전송(transfer)**이 일어나는 단 한 순간: **VALID와 READY가 둘 다 1인 클럭 엷지.** 그 엷지에서 정확히 1개의 데이터가 넘어간다. 한쪽이라도 0이면 그 사이클엔 아무 일도 없다(데이터는 그 자리에 머문다).

한 줄로: **transfer = VALID & READY (rising edge에서 샘플).**

2.3 파형으로 차근차근

D 는 데이터, * 는 전송이 일어나는 사이클이다.



한 박자씩 읽자.

- **t1**: 송신이 VALID=1로 D0를 내놓았지만, 수신이 아직 READY=0. → **전송 없음**. D0는 t2까지 그대로 유지된다(버려지지 않는다). 이게 핵심 규칙 1.
- **t2**: 이제 READY=1. VALID&READY=1 → **D0 전송(★)**. 송신은 다음 데이터 D1로 넘어간다.
- **t3**: VALID=1(D1), 그런데 READY=0(수신이 한 박자 쉰다). → **전송 없음**. **D1은 유지**. 이게 **backpressure**의 순간이다 — 수신이 ready를 내려서 송신을 세운 것이다.
- **t4**: READY=1 복귀 → **D1 전송(★)**.
- **t5**: VALID=1(D2), READY=1 → **D2 전송(★)**.
- **t6**: 송신이 더 줄 게 없어 VALID=0 → 채널 idle.

2.4 backpressure — 왜 이게 모든 버스의 핵심인가

t3에서 일어난 일이 **backpressure(역압)** 다. 수신이 처리할 여력이 없을 때 READY를 내리면, 그 신호가 송신을 뒤로 밀어 멈춰 세운다. 송신은 데이터를 버리지 않고 기다린다.

이게 왜 결정적인가? 만약 backpressure가 없다면(수신이 무조건 받아야 한다면), 모든 수신자는 **최악의 순간 폭주하는 데이터를 모두 흡수할 무한 버퍼**를 가져야 한다 — 비현실적이다. backpressure는 "받을 수 있을 때만 보내라"는 한마디로 이 문제를 푼다. 흐름 제어(flow control)의 가장 단순하고 강력한 형태다.

여기서 SoC 통합 엔지니어가 반드시 아는 두 가지 **함정**:

1. **VALID는 READY를 기다리면 안 된다(조합 의존 금지의 방향성)**. AXI 같은 표준은 "**VALID가 1로 올라온 뒤에는, 전송이 끝날 때까지 VALID를 내리거나 데이터를 바꾸면 안 된다**"고 못 박는다. 즉 송신은 일단 약속하면 끝까지 지켜야 한다. 반대로 READY는 VALID를 보고 결정해도 된다. 이 **비대칭**이 deadlock을 막는다(§6).
2. **조합 루프(combination loop) 금지**. "내 VALID = 상대 READY", "상대 READY = 내 VALID" 식으로 양쪽이 서로의 핸드셰이크 신호를 같은 사이클에 조합으로 참조하면 루프가 생긴다. 그래서 한쪽(보통 송신의 VALID)은 상대 신호와 **독립적으로** 결정되도록 설계한다.

면접 한 줄. "valid/ready는 송수신이 둘 다 준비된 엣지에서만 정확히 한 번 전송을 일으켜 loss와 duplication을 원천 차단하고, ready를 내리는 backpressure로 무한 버퍼 없이 흐름을 제어합니다. 이게 모든 온칩 버스의 공통 원자입니다."

3. AMBA 패밀리의 진화 — APB → AHB → AXI

ARM의 **AMBA(Advanced Microcontroller Bus Architecture)**는 온칩 버스의 사실상 산업 표준이다. 한 번에 만들어진 게 아니라, **필요가 생길 때마다 진화**했다. 그 진화의 동기를 따라가면 각 프로토콜이 "왜 이렇게 생겼는지"가 저절로 보인다.

3.1 APB — 단순 저속 주변장치를 위한 최소 버스

가장 단순한 요구부터. 칩 안에는 UART, GPIO, 타이머, 각종 설정 레지스터(SFR)처럼 **느리고, 가끔 접근하고, 고대역이 전혀 필요 없는** 주변장치가 많다. 이들에 고성능 버스를 붙이는 건 낭비다(면적·전력·복잡도).

APB(Advanced Peripheral Bus) 는 이 요구에 답한다. 특징: - **파이프라인 없음, burst 없음**. 한 번에 한 transfer. - 2-페이지(setup → enable/access)로 도는 단순한 상태기계. - 신호가 적다(PSEL, PENABLE, PWRITE, PADDR, PWDATA, PRDATA, PREADY). 설계·검증이 쉽다. - **전형적 용도**: 레지스터(SFR) 접근. 메모리 컨트롤러나 PHY의 **설정 레지스터**를 호스트가 읽고 쓰는 경로가 보통 APB다. (당신이 삼성에서 다룬 "SFR"이 바로 이런 레지스터 — 05의 CDC 맥락에서도 SFR가 등장했다.)

APB는 "단순함이 곧 미덕"인 영역의 답이다. 더 빠를 필요가 없으니 더 복잡할 이유가 없다.

3.2 AHB — 단일 마스터 파이프라인 버스

다음 요구: CPU가 메모리/캐시처럼 **자주, 빠르게** 접근하는 고대역 경로. APB로는 느려서 안 된다. 여기서 두 가지 아이디어가 추가된다.

1. **파이프라인(주소 페이즈와 데이터 페이즈 분리)**. 현재 transfer의 데이터가 흐르는 동안, **다음 transfer의 주소**를 미리 내보낸다. 주소-데이터 2단 파이프라인으로 throughput을 끌어올린다.
2. **burst**. 연속된 주소에 대한 다중 transfer를 한 묶음으로 보낸다(예: 캐시 라인 채우기 = 4/8 word 연속 읽기). 매번 주소를 다시 안 줘도 된다.

AHB(Advanced High-performance Bus) 의 한계도 명확하다: - **단일 transaction in-flight 중심**. 기본적으로 한 transfer가 끝나야(또는 파이프라인 깊이만큼만) 다음이 진행된다. 슬레이브가 느리면(HREADY를 늘리면) 버스 전체가 그만큼 기다린다 — **read와 write가 같은 채널을 공유하기** 때문이다. - 다중 마스터는 가능하지만 **공유버스 + 중재** 구조라, 마스터가 늘면 경쟁이 심해진다.

AHB는 "단일 고성능 경로"엔 충분하지만, "수많은 마스터가 동시에, 지연이 긴 메모리를 향해, 순서를 섞어가며 고대역으로" 접근하는 현대 SoC엔 부족하다. 그 부족을 메우려고 나온 게 AXI다.

3.3 AXI — 고성능 인터넥트의 표준

AXI(Advanced eXtensible Interface) 는 AMBA의 고성능 답이다. AHB의 한계를 정면으로 깨는 네 가지 설계 결정으로 이루어진다. 하나씩 "왜"를 보자.

(1) 5개 독립 채널 — 읽기와 쓰기를 분리

AXI는 트랜잭션을 **5개의 독립적인 채널**로 쪼갬다. 각 채널은 §2의 valid/ready 핸드셰이크로 독립적으로 흐른다.

채널	약자	방향	역할
Read Address	AR	M→S	"이 주소에서 읽어줘"(주소·길이·ID)
Read Data	R	S→M	읽은 데이터(+응답)를 돌려줌
Write Address	AW	M→S	"이 주소에 쓸게"(주소·길이·ID)
Write Data	W	M→S	실제 쓸 데이터
Write Response	B	S→M	"썼다"는 완료 응답

(M=마스터, S=슬레이브.) 왜 5개로 쪼개나? **읽기와 쓰기가 서로를 기다리지 않게** 하려는 것이다. AHB에선 read·write가 한 채널을 공유해 한쪽이 다른 쪽을 막았다. AXI에선 읽기 트랜잭션과 쓰기 트랜잭션이 **물리적으로**

다른 선을 타고 동시에 진행된다. 게다가 주소(AR/AW)와 데이터(R/W)도 분리돼 있어, 주소를 미리 쏘아붙고 데이터는 뒤따라 흐르게 할 수 있다.

(2) Outstanding transaction — 응답을 기다리지 않고 여러 요청을 던진다

메모리는 **지연(latency)**이 길다. 요청을 보내고 데이터가 돌아오기까지 수십~수백 사이클이 걸릴 수 있다. 만약 "한 요청을 보내고 응답이 올 때까지 다음 요청을 못 보낸다"면, 그 긴 지연 동안 버스는 놀고 throughput은 바닥난다.

outstanding transaction은 이걸 깬다. 마스터는 **응답이 오기 전에 여러 개의 요청을 연달아 던질 수 있다**. 요청 A의 데이터가 돌아오기를 기다리는 동안 요청 B, C, D를 이미 발사한 상태 — 이걸 "outstanding이 4개"라고 한다. 메모리의 긴 지연이 여러 요청으로 **숨겨진다(latency hiding)**. 고대역 메모리 시스템의 핵심 기법이다.

(3) Burst — 한 번 주소로 여러 데이터

AXI도 burst를 지원한다(AHB보다 정교하게). 한 번의 AR/AW로 길이(예: 1~256 beat)·크기·타입(INCR 증가주소 / WRAP 캐시라인 / FIXED 고정주소)을 지정하면, 그 묶음만큼 데이터가 연달아 흐른다. 캐시 라인 채우기, DMA의 큰 블록 전송이 전형이다.

(4) ID 기반 out-of-order completion

이게 AXI의 가장 정교한 부분이다. 각 트랜잭션에는 **ID(AxID)**가 붙는다. 슬레이브는 여러 outstanding 요청을 **들어온 순서와 다른 순서로 완료해도 된다** — 단, **같은 ID끼리는 순서를 지킨다**.

왜 out-of-order가 필요한가? 메모리 시스템은 요청마다 응답 속도가 다르다. 어떤 주소는 이미 열린 row에 있어 빠르고(row hit), 어떤 주소는 새 row를 열어야 해 느리다(row miss). 빠른 응답을 굳이 느린 응답 뒤에서 기다리게 하면 손해다. ID를 붙여두면 슬레이브는 **준비되는 대로 응답을 돌려보내고**, 마스터는 ID로 "이건 아까 던진 그 요청의 응답"이라고 짝을 맞춘다.

순서 규칙(ordering)의 정확한 표현: - 같은 AxID를 가진 트랜잭션들끼리는 발행 순서대로 완료돼야 한다(in-order within same ID). - 서로 다른 ID 사이엔 순서 보장이 없다(out-of-order allowed).

이 "ID 기반 ordering"이 §8의 검증에서 SVA로 잡아야 할 핵심 규칙이 된다. 그리고 §6의 deadlock과도 직결된다 (잘못된 ordering 가정이 교착을 만든다).

AXI 변종 한 줄 정리

- **AXI4**: 위 풀 기능(burst up to 256).
- **AXI4-Lite**: burst·복잡 기능 제거한 단순 버전. 레지스터 접근용(APB 대안).
- **AXI-Stream(AXIS)**: 주소 없는 순수 데이터 스트림(예: 미디어·DMA 파이프). TVALID/TREADY/TDATA/TLAST. 주소 개념이 필요 없을 때.
- **ACE / CHI**: 캐시 일관성(coherency)까지 확장. 멀티코어·멀티클러스터에서 캐시 간 일관성을 버스 레벨에서 관리. JD의 "AC"가 가리키는 coherency 영역이 여기다(ACE = AXI Coherency Extensions).

3.4 한 표로 보는 진화

	APB	AHB	AXI
동기	단순 저속 주변장치	단일 고성능 경로	고성능 멀티마스터 인터커넥트
채널	단일, 2-페이지	주소/데이터 파이프라인(공유)	5채널 독립(AR/R/AW/W/B)
read/write	분리 안 됨	공유 채널	완전 분리(동시 진행)
burst	없음	있음	있음(정교, 256 beat)
outstanding	없음	제한적	여러 개 in-flight
순서	자명	in-order	ID별 in-order, 전체 out-of-order
전형 용도	SFR/레지스터	캐시·간단 고대역	메모리·DMA·가속기 고대역

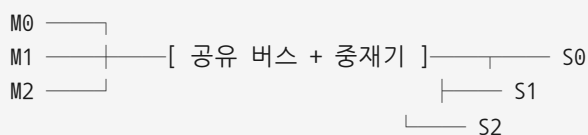
면접 한 줄(AXI > AHB). "AHB는 read/write가 한 채널을 공유해 서로를 막고, 한 트랜잭션이 끝나야 다음이 진행되니 긴 메모리 지연 동안 버스가 놓입니다. AXI는 5채널 분리로 읽기·쓰기를 동시에 흘리고, outstanding으로 응답 전에 여러 요청을 던져 지연을 숨기며, ID로 out-of-order 완료를 허용해 빠른 응답이 느린 응답에 막히지 않게 합니다. 고대역 메모리 시스템에 본질적으로 맞습니다."

4. 인터커넥트 토폴로지 — 공유버스 / Crossbar / NoC

§2~§3이 "두 IP 사이의 규약"이었다면, 이제 **여러 마스터·여러 슬레이브를 어떻게 엮느냐**의 구조 문제다. 마스터 M개, 슬레이브 N개를 잇는 방식엔 세 가지 큰 갈래가 있고, 각각 대역폭·확장성·면적의 trade-off가 다르다.

4.1 공유버스(shared bus) — 모두가 한 도로

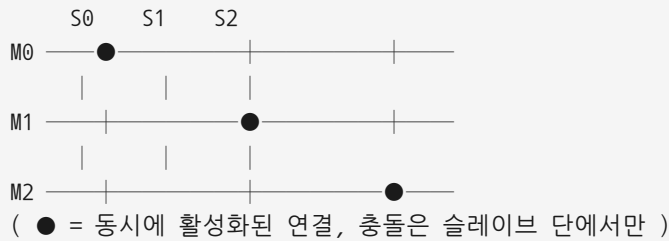
가장 단순. 모든 마스터·슬레이브가 **하나의 공통 버스선**에 붙는다. 한 순간엔 한 마스터만 버스를 점유한다(중재로 결정, §5).



- **장점:** 배선·면적 최소. 단순.
- **단점:** 한 순간 한 transfer만 → **대역폭이 마스터 수로 나뉜다**. 마스터가 늘수록 경쟁·대기 폭증. 긴 배선 → 타이밍 closure 어려움. **확장성 최악**.
- 초기 AHB 시스템의 기본형. 소규모 SoC엔 충분.

4.2 Crossbar(crossbar switch) — 모든 쌍에 전용 경로

확장성 문제를 정면으로 푼다. **각 마스터에서 각 슬레이브로 가는 경로를 동시에 열 수 있게 한다**. M0가 S0와, 동시에 M1이 S1과 통신 — 서로 안 막힌다(목적지가 다르면).



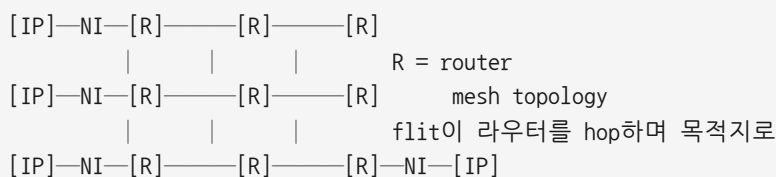
- **장점: 병렬 대역폭.** 서로 다른 마스터-슬레이브 쌍이 동시에 풀스피드로 통신. AXI 인터넥트의 전형적 구현이 crossbar(또는 그 부분집합)다.
- **단점: 연결점이 $M \times N$ 으로 늘어 면적·배선이 폭증한다.** 마스터·슬레이브가 많아지면 crossbar는 감당이 안 된다. 충돌은 같은 슬레이브를 동시에 노릴 때만 발생하고, 그 지점엔 여전히 중재가 필요하다.

대부분의 상용 AMBA "인터넥터" IP는 풀 crossbar와 공유버스 사이의 **부분 crossbar(partial crossbar)** 다 — 필요한 경로만 풀로 열고, 트래픽이 적은 쪽은 공유해 면적을 아낀다.

4.3 NoC(Network-on-Chip) — 칩 안에 패킷 네트워크를 깐다

마스터·슬레이브가 수십~수백 개로 늘면 crossbar의 $M \times N$ 면적이 폭발한다. 여기서 발상을 바꾼다: **칩 안에 작은 컴퓨터 네트워크(인터넷)를 깐다.**

NoC의 핵심 구성요소(인터넷의 축소판): - **packet / flit**. 트랜잭션을 **패킷**으로 만들고, 패킷을 다시 작은 고정 단위 **flit(flow control digit)** 으로 쪼갬다. flit이 네트워크를 흐르는 최소 단위다. 보통 head flit(헤더·목적지) + body flit(데이터) + tail flit(끝)로 구성. - **router(switch)**. 격자 교차로마다 라우터가 있어, 들어온 flit을 목적지 방향으로 보낸다. 라우팅 알고리즘(예: 2D mesh의 XY 라우팅 — 먼저 X축, 그다음 Y축으로)이 경로를 정한다. - **link**. 라우터 사이를 잇는 배선. - **NIU / NI(Network Interface Unit)**. IP의 버스 인터페이스(AXI 등)를 NoC 패킷으로 **포장·해체**하는 어댑터. IP는 여전히 AXI로 말하고, NI가 그걸 패킷으로 바꿔 네트워크에 태운다. (그래서 NoC는 AXI를 대체하는 게 아니라, AXI 트랜잭션을 **운반하는 전송계층**이다.) - **virtual channel(VC)**. 하나의 물리 링크를 논리적으로 여러 채널로 쪼갬다. 왜? (a) head-of-line blocking 완화 — 한 패킷이 막혀도 다른 VC의 패킷은 통과, (b) **deadlock 회피**(§6) — 클래스를 분리해 순환 의존을 끊는다. - **topology**. 라우터를 어떻게 배치하나 — **mesh(격자)**, **torus(격자+가장자리 연결)**, **ring(고리)**, **tree** 등. mesh가 가장 흔하다(확장성·규칙성).



- **장점: 확장성이 본질.** IP가 늘어도 라우터를 격자로 추가하면 된다(면적이 $M \times N$ 이 아니라 노드 수에 선형). 배선이 짧은 구간으로 쪼개져 타이밍 closure 유리. 대역폭이 분산 라우팅으로 확장.
- **단점: 라우터·NI·버퍼의 오버헤드**(면적·전력·지연). 패킷화/디패킷화 지연. 소규모엔 과잉.

4.4 언제 NoC로 가나 — 결정 기준

면접에서 "언제 crossbar 대신 NoC를 씁니까"를 물으면, 다음 3가지로 답한다:

1. **IP(노드) 수.** 마스터·슬레이브가 적으면(수 개~십수 개) crossbar가 면적·지연 면에서 유리. 수십 개 이상으로 늘면 crossbar의 $M \times N$ 이 감당 안 돼 NoC로 간다.

2. **물리적 거리·타이밍.** 큰 다이에서 한쪽 끝의 IP가 반대쪽 메모리에 접근하려면 긴 배선이 필요하다. 긴 단일 배선은 타이밍 closure가 지옥이다. NoC는 라우터 사이의 **짧은 구간**으로 쪼개 closure를 쉽게 한다(파이프라인 라우팅).
3. **트래픽 다양성·QoS.** 여러 종류의 트래픽(저지연 CPU, 고대역 GPU, 실시간 디스플레이)이 섞이면, VC·우선순위로 트래픽을 클래스별로 관리하는 NoC가 유리.

한 줄 trade-off. "공유버스는 면적 최소·확장성 최악, crossbar는 병렬 대역폭이지만 $M \times N$ 면적 폭증, NoC는 면적이 노드 수에 선형이고 확장성·타이밍에 유리하나 라우터 오버헤드가 있습니다. 노드가 많고 다이가 크면 NoC, 적으면 crossbar입니다."

5. 중재(arbitration) — 동시 요청을 누구에게 줄 것인가

§4에서 봤듯, 같은 자원(공유버스, 또는 crossbar의 같은 슬레이브 포트)을 여러 마스터가 동시에 원하면 충돌한다. 누구에게 먼저 줄지 정하는 게 **중재(arbitration)** 다. 중재기의 설계가 시스템의 **공정성·지연·deadlock-free**를 좌우한다.

5.1 Fixed priority — 가장 단순, starvation 위험

각 마스터에 고정 우선순위를 매긴다. $M_0 > M_1 > M_2$ 면, M_0 가 요청하는 한 M_0 가 항상 이긴다.

- **장점:** 구현 단순, 결정론적. 정말 급한 트래픽(예: refresh, 인터럽트)엔 적합.
- **치명적 단점: starvation(기아).** 높은 우선순위 마스터가 계속 요청하면, 낮은 우선순위 마스터는 **영원히 자원을 못 받을 수 있다**. M_0 가 쉬지 않으면 M_2 는 굶는다. 이건 단순 성능 저하가 아니라 **기능적 행 hang**으로 이어질 수 있다.

5.2 Round-robin — 공정성의 기본

순서대로 한 바퀴 돌며 기회를 준다. 방금 자원을 받은 마스터는 **맨 뒤로** 보낸다. M_0 가 받았으면 다음 우선순위는 $M_1 \rightarrow M_2 \rightarrow M_0$ 순으로 회전한다.

요청 패턴: M_0, M_1, M_2 가 매 사이클 모두 요청한다고 하자.
RR 결과 : $M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow M_0 \rightarrow M_1 \rightarrow M_2 \rightarrow \dots$
(아무도 굶지 않는다. 모두 1/3씩 공정하게.)

- **장점: 기아 없음(starvation-free).** 모든 마스터가 유한 시간 안에 반드시 기회를 받는다(bounded latency). 가장 널리 쓰이는 기본 중재.
- **왜 공정성이 중요한가:** 어떤 마스터든 "유한 시간 안에 반드시 진행한다"는 보장(progress guarantee)이 있어야 시스템이 hang하지 않는다. round-robin은 이 보장을 구조적으로 준다.

5.3 Weighted round-robin — 차등 공정성

순수 RR은 모두에게 똑같이 $1/N$ 을 준다. 그런데 트래픽 중요도가 다르면? GPU엔 대역을 많이, 저속 주변장치엔 적게 주고 싶다. **가중치(weight)**를 매겨 RR을 돌린다 — 가중치가 3:1이면 M_0 가 3번 받을 동안 M_1 이 1번 받는다. **여전히 기아는 없으면서(모두 결국 받음)** 대역을 차등 배분한다. QoS의 기본 구현.

5.4 Credit-based flow control — 버퍼를 넘기지 않게

이건 중재이자 흐름 제어다. 특히 NoC와 깊은 파이프라인에서 핵심.

아이디어: 송신자는 수신자의 빈 버퍼 칸 수만큼 "크레딧(credit)" 을 갖고 시작한다. 한 flit/패킷을 보낼 때마다 크레딧을 1 깎는다. 크레딧이 0이면 보내지 않는다. 수신자가 버퍼를 비우면(처리 완료) "크레딧 1 반환" 신호를 보내 송신자의 크레딧을 회복시킨다.

초기: 수신 버퍼 4칸 → 송신 크레딧 = 4

send → credit 3 → send → 2 → send → 1 → send → 0 (이제 멈춤)

수신이 1칸 처리 → "credit 반환" → 송신 크레딧 1 → 다시 send 가능

- 왜 valid/ready 대신/위에 크레딧인가: valid/ready는 한 사이클 거리의 즉각 backpressure다. 그런데 송신과 수신이 여러 사이클 떨어져 있으면(긴 배선, 파이프라인된 NoC 라우터 사이), ready가 송신에 도달하는 데 시간이 걸려 즉각 멈출 수 없다. 크레딧은 "상대 버퍼에 빈자리가 있음을 미리 아는" 방식이라 거리가 멀어도 버퍼 오버플로를 정확히 막는다. 그래서 NoC의 표준 흐름 제어가 credit-based다.
- credit 흐름 제어는 deadlock 회피의 한 축이기도 하다(§6) — 버퍼가 보장되니 채널이 영원히 막히는 상황을 구조적으로 줄인다.

5.5 QoS(Quality of Service)

큰 인터넥트는 트래픽마다 QoS 클래스(우선순위·대역 보장·지연 한계)를 부여한다. AXI에도 QoS 신호(AxQOS)가 있다. 중재기는 이 값을 보고 우선순위를 조정한다. 예: 디스플레이 같은 실시간(deadline) 트래픽엔 높은 우선순위로 지연을 보장하고, 배치성 DMA엔 대역만 보장. 공정성(아무도 안 굶음) + 차등(중요한 건 우대) 을 동시에 만족시키는 게 QoS 중재의 목표다.

면접 한 줄(round-robin이 필요한 이유). "fixed priority는 단순하지만 높은 우선순위가 자원을 독점하면 낮은 마스터가 영원히 못 받는 starvation이 납니다. round-robin은 방금 받은 마스터를 맨 뒤로 보내 모두에게 유한 시간 안에 기회를 보장하므로(starvation-free), 시스템 hang을 구조적으로 막습니다. 중요도 차이는 weighted RR이나 QoS로 줍니다."

6. 위험 — deadlock / livelock / starvation

중재와 채널 구조를 잘못 설계하면 시스템이 멈춘다. 세 가지 멈춤을 정확히 구분하고, 각각을 어떻게 막는지 알아야 한다. 이건 면접 단골이고, 동시에 SoC 통합/검증에서 가장 무서운 버그류다.

6.1 세 가지 멈춤의 정의

	무엇	비유
deadlock(교착)	서로가 서로의 자원을 기다려 아무도 진행 못 함 . 영구 정지.	사거리에서 네 차가 서로 앞차를 기다리며 영원히 안 움직임
livelock(활락)	신호는 바쁘게 바뀌는데 유효한 진행이 없음 . 죽진 않았는데 일이 안 됨.	좁은 복도에서 두 사람이 계속 같은 방향으로 비키다 영원히 못 지나감
starvation(기아)	시스템 전체는 진행하지만 특정 주체만 영원히 못 받음 .	끼어들기 차 때문에 한 차선이 영원히 못 빠져나감

6.2 deadlock은 어떻게 생어나 — 순환 의존과 짝 찬 채널

deadlock의 교과서적 4조건(Coffman): 상호배제, hold-and-wait, 비선점, **순환 대기(circular wait)**. 온칩 버스에서 이게 구체적으로 어떻게 나타나는가:

전형 시나리오 1 — 순환 의존(circular dependency). - 마스터 A가 슬레이브 X를 점유한 채 슬레이브 Y의 응답을 기다린다. - 마스터 B가 슬레이브 Y를 점유한 채 슬레이브 X의 응답을 기다린다. - A는 Y가 풀리길, B는 X가 풀리길 기다린다 → **순환** → 둘 다 영원히 멈춤.

전형 시나리오 2 — 짝 찬 채널 + 채널 결합(read/write coupling). - AXI에서 **write 응답(B)이 read 데이터(R) 채널의 진행에 의존**하도록 잘못 설계하면(또는 슬레이브가 write를 완료하려면 먼저 read를 받아야 하는데 read 채널이 막혀 있으면), 두 채널이 서로를 기다리며 교착. 이래서 AXI는 **채널을 독립적으로** 두고 "한 채널의 진행이 다른 채널의 핸드셰이크에 의존하면 안 된다"는 규칙을 둔다.

전형 시나리오 3 — VALID가 READY를 기다림. - §2.4에서 본 비대칭 위반. 송신 A의 VALID가 상대의 READY가 올라오길 기다리고, 상대의 READY는 A의 VALID가 올라오길 기다리면 → 둘 다 0에서 영원히 정지. 이래서 "VALID는 READY와 독립적으로 결정한다"는 규칙이 deadlock-free의 핵심이다.

6.3 어떻게 막나 — 4가지 구조적 방어

1. **채널 분리(independent channels).** read와 write, 주소와 데이터를 물리적으로 분리해 한쪽이 다른 쪽을 막지 못하게 한다(AXI 5채널의 근본 이유). 한 채널의 진행을 다른 채널의 핸드셰이크에 결합시키지 않는다.
2. **ordering 규칙 준수.** AXI의 "같은 ID는 in-order, 다른 ID는 out-of-order" 규칙을 지키면, 서로 다른 트랜잭션이 부당하게 서로를 기다리는 순환을 막는다. ID를 함부로 재사용하거나 ordering을 잘못 가정하면 교착이 생긴다.
3. **순환 의존 제거(dependency ordering).** 자원에 전역 순서를 부여하고 항상 그 순서로만 획득하게 하면(예: 항상 X→Y 순서로 요청), circular wait가 원천적으로 불가능해진다. NoC에선 라우팅 알고리즘 자체를 deadlock-free로 설계(예: XY 라우팅은 turn 제약으로 순환 경로를 막는다).
4. **credit/VC로 채널 보장.** credit 기반 흐름 제어는 버퍼가 보장되니 "채널이 영원히 짝 차서 막힘"을 줄인다. virtual channel은 트래픽 클래스를 분리해, 한 클래스의 정체가 다른 클래스를 막지 않게 하고 순환 의존을 끊는다(예: 요청 VC와 응답 VC 분리 → 응답이 요청에 막혀 생기는 protocol deadlock 방지).

6.4 livelock·starvation 방어

- **livelock**: 무작정 양보·재시도를 반복하지 않게, **결정론적 우선순위(타이브레이커)** 나 backoff에 무작위/증가 지연을 둔다. 중재에 명확한 승자 규칙을 뒤 "서로 비키다 영원히"를 막는다.
- **starvation**: §5의 round-robin/weighted/QoS로 **bounded latency**(유한 시간 진행 보장)를 준다. fixed priority의 무한 독점을 구조적으로 차단.

면접 한 줄(deadlock 방어). "deadlock은 자원의 순환 대기로 생깁니다. 막는 방법은 (1) read/write-주소/데이터 채널을 독립으로 뒤 한쪽이 다른 쪽을 못 막게 하고, (2) AXI의 ID별 ordering 규칙을 지키며, (3) 자원 획득에 전역 순서를 줘 순환을 원천 차단하고, (4) NoC에선 virtual channel로 요청·응답 트래픽을 분리하고 credit으로 버퍼를 보장하는 것입니다. 그리고 검증에선 이 deadlock-free 속성을 SVA/formal로 잡습니다."

7. 클럭·전압 도메인을 가로지르는 버스 — Async Bridge

버스가 한 클럭·전압 도메인 안에만 머무는 일은 드물다. 고대역 코어 도메인에서 출발한 트랜잭션이 저속 주변장치 도메인으로, 또는 한 전압 섬에서 다른 전압 섬으로 건너간다. JD의 "Top Bus / Async Bridge Implementation"이 정확히 이 지점이다.

7.1 async bridge = 버스 위에 올라탄 CDC 문제

버스가 비동기 클럭 경계를 건널 때 쓰는 회로가 **async bridge(비동기 브릿지)** 다. 본질은 **CDC 그 자체**다. 두 도메인의 클럭이 위상 관계가 없으니, 한 도메인의 신호를 다른 도메인이 붙잡는 순간 setup/hold 위반 → **메타스터빌리티** 위험이 생긴다(이 메커니즘의 끝까지의 해설은 05_CDC_RDC_완전이해.md).

버스가 도메인을 건널 때 구체적으로 무엇을 동기화해야 하나: - **핸드셰이크 제어 신호(valid/ready)**: 단일 비트 → 2-FF 동기화(05 참조). valid가 도메인을 건너고, ready가 반대로 건너온다. - **데이터·주소(multi-bit)**: 여러 비트가 동시에 건너면 bit skew로 잘못된 값이 잡힐 수 있다(05의 multi-bit 문제). 그래서 **async FIFO(gray-code 포인터로 양 도메인 안전 동기화)** 또는 **handshake 기반 MUX-hold(데이터를 고정해두고 제어 신호만 안전 전달 후 샘플)**를 쓴다. AXI async bridge의 전형은 채널마다 async FIFO를 두는 것이다.

7.2 왜 버스에서 특히 까다로운가

버스는 단일 신호가 아니라 **여러 채널·여러 비트가 한 트랜잭션으로 묶여** 건넌다. 주소, 데이터, ID, valid, ready, 응답 — 이것들이 **일관되게(coherently)** 건너야 한다. 데이터는 건넜는데 valid가 한 박자 어긋나 건너면 트랜잭션이 깨진다. 그래서 async bridge 설계는 "각 비트의 CDC"를 넘어 "트랜잭션 전체의 atomicity를 보존하는 CDC"가 된다. async FIFO가 답인 이유다 — 한 entry 안에 트랜잭션의 모든 필드를 묶어 gray-pointer로 한꺼번에 안전하게 건넌다.

7.3 다중 전압 도메인 통과 — level shifter / isolation

클럭만 다른 게 아니라 **전압 도메인**도 다를 수 있다(전력 절약 위해 블록마다 전압을 다르게, 또는 끄기도). 버스가 전압 섬을 건널 때: - **level shifter**: 낮은 전압(예: 0.6V)의 신호를 높은 전압(예: 0.9V) 도메인이 올바르게 인식하도록 레벨 변환. - **isolation cell**: 한 전압 도메인이 꺼져 있을 때(power gating), 그 출력이 부정확한 floating 값으로 살아있는 도메인을 오염시키지 않도록 안전한 값(0 또는 1)으로 고정. - 이것들은 UPF/CPF(전력 의도 명세)로 정의되고, 통합 시 자동 삽입·검증된다.

SoC 통합 엔지니어로서 당신의 포인트. "Top에서 IP를 통합할 때 버스가 클럭·전압 도메인을 건너는 모든 지점이 async bridge(=CDC)이자 power domain crossing입니다. 저는 삼성에서 이 도메인 경계를 CDC/RDC sign-off로 검증한 사람이라, async bridge 통합과 그 검증이 제 핵심 업무와 정확히 겹칩니다." (CDC 깊이는 05 참조.)

8. 검증 관점 — 버스 프로토콜을 어떻게 증명하나 (UVM VIP + SVA)

여기가 당신의 무대다. 버스를 "설계한다"보다 "spec대로 도는지 증명한다"가 당신의 강점이고, 그 증명 대상이 지금까지 본 모든 규칙이다. (검증 방법론 전반은 07_검증_DFT_MBIST_완전이해.md 참조 — 여기선 버스에 특화해 연결한다.)

8.1 두 무기 — UVM VIP와 SVA

버스 프로토콜 검증의 두 축:

1. **UVM VIP(Verification IP) / Agent.** 표준 버스(AXI 등)는 보통 상용 UVM VIP가 있다 — 마스터/슬레이브를 흉내 내는 BFM(bus functional model), 자극을 만드는 sequence, 프로토콜 체커, 커버리지가 패키지로 묶인 것. SoC 검증 환경에 이 VIP를 꽂아 트랜잭션을 쏘아붙고 응답을 본다.
2. **SVA(SystemVerilog Assertions).** 프로토콜 규칙을 시간 관계식(temporal assertion) 으로 직접 박아, 시뮬레이션 내내 매 사이클 위반을 감시한다. 07 에서 본 "전수 vs 본 것만"의 관점에서, SVA는 시뮬레이션 동안 모든 사이클의 규칙 준수를 자동 감시하고, formal(JasperGold 등)에 걸면 입력 전수로 그 속성이 항상 참인지 증명할 수 있다.

8.2 무엇을 assertion으로 잡나 — 버스 규칙의 SVA화

§2~§6에서 본 규칙들이 그대로 assertion이 된다. 대표 예:

(a) **handshake 안정성 — VALID 유지 규칙.** "VALID가 1이 된 뒤 전송(VALID&READY)이 일어나기 전까지, VALID는 1을 유지하고 DATA는 바뀌면 안 된다."

```
// VALID가 뒀는데 아직 전송 안 됐으면(READY=0), 다음 사이클에도 VALID 유지
property p_valid_stable;
  @(posedge clk) disable iff (!rst_n)
    (VALID && !READY) |>= VALID; // |>= : 다음 사이클에 반드시 성립
endproperty
assert property (p_valid_stable);

// 전송 전까지 DATA 불변
property p_data_stable;
  @(posedge clk) disable iff (!rst_n)
    (VALID && !READY) |>= $stable(DATA);
endproperty
assert property (p_data_stable);
```

(b) **outstanding / ID ordering.** "같은 AxID로 발행된 트랜잭션의 응답은 발행 순서대로 돌아와야 한다." → ID별 큐(scoreboard)를 두고, 응답 순서가 발행 순서와 같은지 검사. (이건 단순 1-property로 안 끝나고 scoreboard + assertion 조합으로 구현.)

(c) 응답 완결성(no orphan / no duplicate). "모든 AR(읽기 주소)에는 정확히 그 길이만큼의 R 데이터가 따라오고, 마지막에 RLAST가 한 번 뜬다." / "모든 AW+W에는 정확히 하나의 B 응답이 온다." → outstanding 카운터로 요청과 응답의 1:1(혹은 1:burst-len) 짝을 검사.

(d) deadlock/liveness — eventually 속성. "VALID가 떴으면 언젠가는 반드시 READY가 떴서 전송이 일어난다 (영원히 막히지 않는다)." → liveness property(s_eventually). formal로 잡으면 deadlock 가능성을 전수 증명할 수 있다.

```
property p_no_deadlock;
  @(posedge clk) disable iff (!rst_n)
  VALID !-> s_eventually (VALID && READY); // 언젠가 반드시 전송
endproperty
assert property (p_no_deadlock);
```

8.3 "Custom UVM Agent를 spec에서" — JD의 진짜 의미

JD에 "custom UVM agent"가 나오는 이유: 표준 AXI에는 상용 VIP가 있지만, 사내에서 새로 만든 버스나 커스텀/변형 프로토콜(예: HBM 내부 채널, 컨트롤러-PHY 사이 사내 규약)에는 VIP가 없다. 그러면 spec 문서를 읽고 직접 UVM agent를 만들어야 한다 — driver(신호를 토글), monitor(트랜잭션을 관찰·복원), sequencer, 그리고 그 프로토콜의 규칙을 담은 SVA 체커까지.

이게 당신의 강점이 빛나는 지점이다. "spec → 검증 환경"으로 옮기는 능력 — handshake·ordering·outstanding·응답 완결성을 assertion으로 형식화하는 능력 — 이 바로 그 일이다. 07 에서 다진 UVM/SVA 역량이 그대로 버스 검증으로 이어진다.

면접 한 줄(버스 검증). "표준 버스는 UVM VIP로, 사내 커스텀 버스는 spec을 읽고 직접 UVM agent(driver/monitor/scoreboard)와 SVA 체커를 만들어 검증합니다. handshake 안정성·ID별 ordering·outstanding 완결성·deadlock-free를 SVA로 형식화하고, 결정적 속성은 formal로 전수 증명합니다. 이게 제가 삼성에서 sign-off 방법론으로 해온 일과 정확히 같은 걸입니다."

9. 메모리·HBM 맥락 — 이 모든 것이 어디서 일어나나

지금까지의 추상이 메모리에서 구체적으로 어디에 박히는지 못 박는다. JD의 "HBM SoC Design/검증", "IP Integration", "Chiplet Interface"가 가리키는 실제 지점들이다.

9.1 HBM Base Die — 작은 SoC의 IP integration

HBM은 여러 DRAM 코어 다이를 쌓고, 맨 아래 **base die(로직 다이)** 에서 외부와 통신한다. 이 base die가 작은 SoC다: - 외부(호스트/인터포저)에서 들어온 채널 트래픽을 받고, - 내부의 테스트·리페어 로직 (MBIST/redundancy, 07 참조), 각종 제어 IP를 엮고, - TSV(09 참조)를 통해 위층 DRAM 다이로 명령·데이터를 분배한다.

이 IP들을 엮는 **내부 인터커넥트**가 바로 §3~§5의 대상이다. HBM 세대가 올라가며(HBM3→HBM4) base die의 디지털 복잡도가 커지고, "IP integration"의 비중이 커진다. JD의 "On-Chip Interconnector"가 여기 직결.

9.2 메모리 컨트롤러 — AXI 프론트엔드 + 내부 채널 + PHY 백엔드

DRAM 컨트롤러의 구조를 버스 관점에서 보면: - **프론트엔드(host side)**: 보통 **AXI 슬레이브 포트**. CPU·가속기가 AXI로 읽기/쓰기 요청을 던진다. 여기서 §3의 outstanding·ID·QoS가 그대로 쓰인다(여러 요청 in-flight, ID로 응답 매칭, QoS로 우선순위). - **내부**: 요청을 받아 row/bank를 고려해 **스케줄링·리오더링**한다(row hit를 모아 효율). 이게 §3.3의 "out-of-order completion"이 실제로 쓰이는 이유 — 컨트롤러가 응답 순서를 바꿔 돌려줄 수 있으니, AXI ID 매칭이 필수다. - **백엔드(PHY side)**: 스케줄된 명령을 DRAM PHY 인터페이스로 내보낸다(09의 PHY·타이밍 영역). 컨트롤러-PHY 사이도 사내 규약(예: DFI 계열)으로 엮이고, 클럭 도메인이 갈리면 §7의 async bridge가 들어간다.

9.3 칩렛(chiplet) — on-chip 인터커넥트가 패키지를 건너는 순간

여기서 on-chip과 off-chip의 경계가 만난다. 칩렛 시대엔 하나의 큰 다이를 여러 작은 다이(칩렛)로 쪼개 패키지 안에서 잇는다. 그 칩렛 간 인터페이스가 JD의 "**Chiplet Interface(UCIe, OpenHBI, BoW)**" 다.

- 칩렛 안의 IP는 여전히 AXI/NoC 같은 **on-chip 인터커넥트**로 말한다.
- 칩렛 경계를 건널 때는 die-to-die(D2D) 물리 인터페이스 — **UCIe, BoW(Bunch of Wires), OpenHBI** — 로 변환돼 옆 칩렛으로 간다. 이건 인터포저/패키지를 지나는 **물리계층** 문제라, PHY·SI 영역이다(off-chip 세부는 09 / 10 참조).
- 구조적으로는 §4.3 NoC의 NI(네트워크 인터페이스)가 패키지 경계까지 확장된 것으로 볼 수 있다 — on-chip 트랜잭션을 D2D 링크용으로 포장·해체. "on-chip 인터커넥트의 논리"는 그대로고, "물리 전송계층"만 패키지로 확장되는 것이다.

브릿지. 즉 on-chip 버스 지식(§2~§8)이 칩렛 경계까지 이어지고, 경계를 넘는 순간 09 (PHY/SI)와 10 (칩렛/패키징)으로 손을 넘긴다. 이 문서는 그 경계 **안쪽**(논리·프로토콜)을 책임진다.

10. 면접 연결 — 30초/1분 골격

각 질문에 (1) 핵심 한 줄 → (2) 1st-principle 한두 단계 → (3) 경험 브릿지 구조로 답한다.

Q1. "valid/ready와 backpressure를 설명해보세요." (30초)

- **핵심**: 송수신이 둘 다 준비된 클럭 엣지에서만 정확히 한 번 전송을 일으키는 2-신호 핸드셰이크입니다 (transfer = VALID & READY).
- **원리**: 이렇게 해야 데이터 loss·중복이 원천 차단됩니다. 수신이 READY를 내리면(backpressure) 송신이 멈춰 기다리므로, 무한 버퍼 없이 흐름을 제어합니다. VALID는 READY와 독립적으로 결정해야 deadlock이 안 생깁니다.
- **브릿지**: 이게 AXI든 NoC든 모든 온칩 버스의 공통 원자라, 어떤 버스 검증이든 이 핸드셰이크 규칙을 SVA로 잡는 데서 출발합니다.

Q2. "AXI가 AHB보다 좋은 점은?" (1분)

- **핵심**: 채널 분리·outstanding·ID 기반 out-of-order, 세 가지입니다.
- **원리**: AHB는 read/write가 한 채널을 공유해 서로를 막고, 한 트랜잭션이 끝나야 다음이 가니 긴 메모리 지연 동안 버스가 놓입니다. AXI는 AR/R/AW/W/B 5채널을 분리해 읽기·쓰기를 동시에 흘리고, outstanding으로

응답 전에 여러 요청을 던져 지연을 숨기며, ID로 빠른 응답이 느린 응답을 추월하게 합니다(같은 ID만 in-order).

- **브릿지:** 메모리 컨트롤러가 row hit를 모아 리오더링하는 게 바로 이 out-of-order이고, 그래서 AXI ID 매칭이 검증의 핵심 포인트가 됩니다.

Q3. "round-robin이 왜 필요합니까?" (30초)

- **핵심:** starvation을 막기 위해서입니다.
- **원리:** fixed priority는 높은 우선순위 마스터가 자원을 독점하면 낮은 마스터가 영원히 못 받아 시스템이 hang합니다. round-robin은 방금 받은 마스터를 맨 뒤로 보내 모두에게 유한 시간 안에 기회를 보장합니다 (bounded latency). 차등이 필요하면 weighted RR이나 QoS를 씁니다.

Q4. "deadlock을 어떻게 막습니까?" (1분)

- **핵심:** 순환 대기를 끊는 것입니다.
- **원리:** (1) read/write·주소/데이터 채널을 독립으로 뒀 한쪽이 다른 쪽을 못 막게 하고(AXI 5채널), (2) "VALID는 READY를 기다리지 않는다"는 핸드셰이크 비대칭을 지키고, (3) 자원 획득에 전역 순서를 줘 circular wait를 원천 차단하고, (4) NoC에선 virtual channel로 요청·응답 트래픽을 분리하고 credit으로 버퍼를 보장합니다.
- **브릿지:** 그리고 이 deadlock-free 속성을 liveness SVA(`VALID |-> s_eventually(VALID&&READY)`)로 잡고, 결정적이면 formal로 전수 증명합니다 — 제가 sign-off에서 해온 형식 검증의 결입니다.

Q5. "버스 프로토콜을 어떻게 검증합니까?" (1분)

- **핵심:** 표준은 UVM VIP, 커스텀은 spec에서 직접 UVM agent + SVA입니다.
- **원리:** handshake 안정성(VALID 유지·DATA 불변), ID별 ordering, outstanding 응답 완결성(no orphan/duplicate), deadlock-free를 SVA로 형식화합니다. 시뮬레이션 내내 매 사이클 감시하고, 결정적 속성은 formal로 입력 전수 증명합니다.
- **브릿지:** 사내 신규 버스나 HBM 내부 채널처럼 VIP가 없는 경우 spec을 읽어 driver/monitor/scoreboard와 체커를 직접 만드는데, 이게 제가 삼성에서 SoC 통합환경과 sign-off로 해온 일과 정확히 같습니다.

Q6. "메모리를 모르는데 SoC 버스 경험이 무슨 소용입니까?" (역공 1분)

- HBM base die도 메모리 컨트롤러도 NAND 컨트롤러도 **내부는 IP가 버스로 엮인 작은 SoC**입니다. 제가 한 IP integration·Top 통합·async bridge 검증·프로토콜 SVA가 그대로 그 안에서 일어나는 일입니다. 메모리 셀은 새로 배우면 되지만, 그걸 둘러싼 로직/Peri의 버스·통합·검증은 제가 이미 4년 해온 영역입니다.

11. 스스로 점검 (정독 후 막힘없이 답할 수 있어야 한다)

1. valid/ready 핸드셰이크에서 전송이 일어나는 정확한 조건은? READY가 0인 사이클에 데이터는 어떻게 되나? (loss가 왜 안 나나)
2. backpressure가 없다면 모든 수신자가 가져야 할 비현실적 조건은 무엇인가?
3. "VALID는 READY를 기다리면 안 된다"는 비대칭 규칙이 왜 deadlock 방지에 핵심인가?
4. APB·AHB·AXI가 각각 어떤 요구 때문에 생겼는지 진화의 동기를 한 문장씩으로 말하라.
5. AXI 5채널(AR/R/AW/W/B)을 각각 방향·역할과 함께 말하고, "왜 5개로 쪼갠나"를 설명하라.

6. outstanding transaction이 메모리의 긴 지연을 어떻게 숨기나? ID 기반 out-of-order에서 보장되는 순서 규칙은 정확히 무엇인가(같은 ID vs 다른 ID)?
7. 공유버스·crossbar·NoC의 대역폭/확장성/면적 trade-off를 말하고, "언제 NoC로 가나"의 기준 3가지를 들라.
8. fixed priority의 starvation은 어떻게 생기고, round-robin은 그걸 어떻게 구조적으로 막나? credit-based flow control이 valid/ready 대신 NoC에서 쓰이는 이유는?
9. deadlock·livelock·starvation을 각각 정의하고, 온칩 버스에서 deadlock이 생기는 시나리오 하나와 그걸 막는 방어 두 가지를 말하라.
10. async bridge가 왜 CDC 문제인지(05 연결), multi-bit 데이터를 도메인 너머로 안전하게 보내는 방법(async FIFO)을 설명하라.
11. 버스 프로토콜 규칙 중 하나(예: VALID 유지)를 SVA property로 직접 써보라. liveness(deadlock-free) 속성은 어떤 연산자로 표현하나?
12. "custom UVM agent를 spec에서 만든다"가 무슨 뜻이고, 그게 왜 당신의 강점과 연결되나?
13. HBM base die와 메모리 컨트롤러를 "버스로 엮인 작은 SoC"로 설명하라. 컨트롤러에서 AXI ID 매칭이 왜 필수인가(리오더링과 연결)?
14. 칩렛 인터페이스(UCIe 등)에서 on-chip 인터커넥트의 논리는 어디까지 가고, 어디서부터 PHY(09) 영역인가?

마무리 한 문장. 온칩 버스는 "준비가 안 맞는 IP들 사이로 누락·교착 없이 공정하게 고대역으로 데이터를 흐르게 하는" 단 하나의 문제를, valid/ready라는 원자에서 AXI·NoC·중재·deadlock 방어까지 한 사술로 풀어낸다. 그리고 그 사술이 spec대로 도는지 UVM/SVA로 증명하는 것이 당신이 이미 가장 잘하는 일이다 — 메모리든 SoC든, IP를 엮고 그 사이를 검증하는 사람의 자리는 늘 비어 있다.